SOFTWARE ENGINEERING
MINI PROJECT

SYSTEM ARCHITECTURE DESIGN

Project Management System

Team 11

Version 1.0 | 26 October 2026

1. Introduction and Purpose

This document describes the proposed high-level architecture for the Project Management System (PMS). It explains the major application layers, feature modules, data-access approach, core request flow, and architecture-level security and quality considerations. It is intended to help all team members understand how the system fits together and to guide implementation consistently.

1.1 Scope

The architecture supports the functionality described in the project requirements and SRS: authentication, project management, task and issue management, Kanban boards, workflow management, estimation, sprint and milestone planning, dashboards, notifications, collaboration, search and filtering, reporting, and activity logging.

1.2 User classes

- Admin: manages users, roles and system settings as defined by the approved requirements.
- Project Manager: manages projects, work planning, visibility and reporting within assigned permissions.
- Team Member: views and updates permitted tasks, collaborates with teammates and tracks assigned work.

1.3 Proposed technology baseline

The following is a proposed baseline based on the team's current architecture discussion. Confirm the final stack before development and keep the SRS, repository and implementation consistent.

Concern	Proposed choice	Purpose / note
Presentation	Django Templates, HTML, CSS and JavaScript	Server-rendered pages for a straightforward initial implementation.
Web application	Python and Django	URL routing, views, forms, sessions, authentication and request validation.
Data access	Django ORM and migrations	Consistent database operations and controlled schema evolution.
Database	PostgreSQL	Relational storage for users, projects, tasks and related records.
Version control	Git and shared GitHub repository	Shared source code, review, history and identifiable contributions.
Automated checks	GitHub Actions	Proposed workflow to run system checks and available tests on pushes and pull requests.

1.4 Architecture principles

- Separation of concerns: keep presentation, request handling, feature rules and persistence responsibilities distinct.
- Server-side validation: enforce required fields, permissions and workflow rules in the application, not only in browser code.
- Incremental delivery: implement and test small end-to-end features before expanding the system.
- Keep the first version simple: use a modular Django application rather than introducing distributed services without a clear need.

2. Logical System Architecture

The proposed design is a layered modular monolith: a single Django application organized into feature modules and backed by a relational database. The feature modules are logical parts of one application, not independent deployed services. All database operations should pass through the application data-access layer.

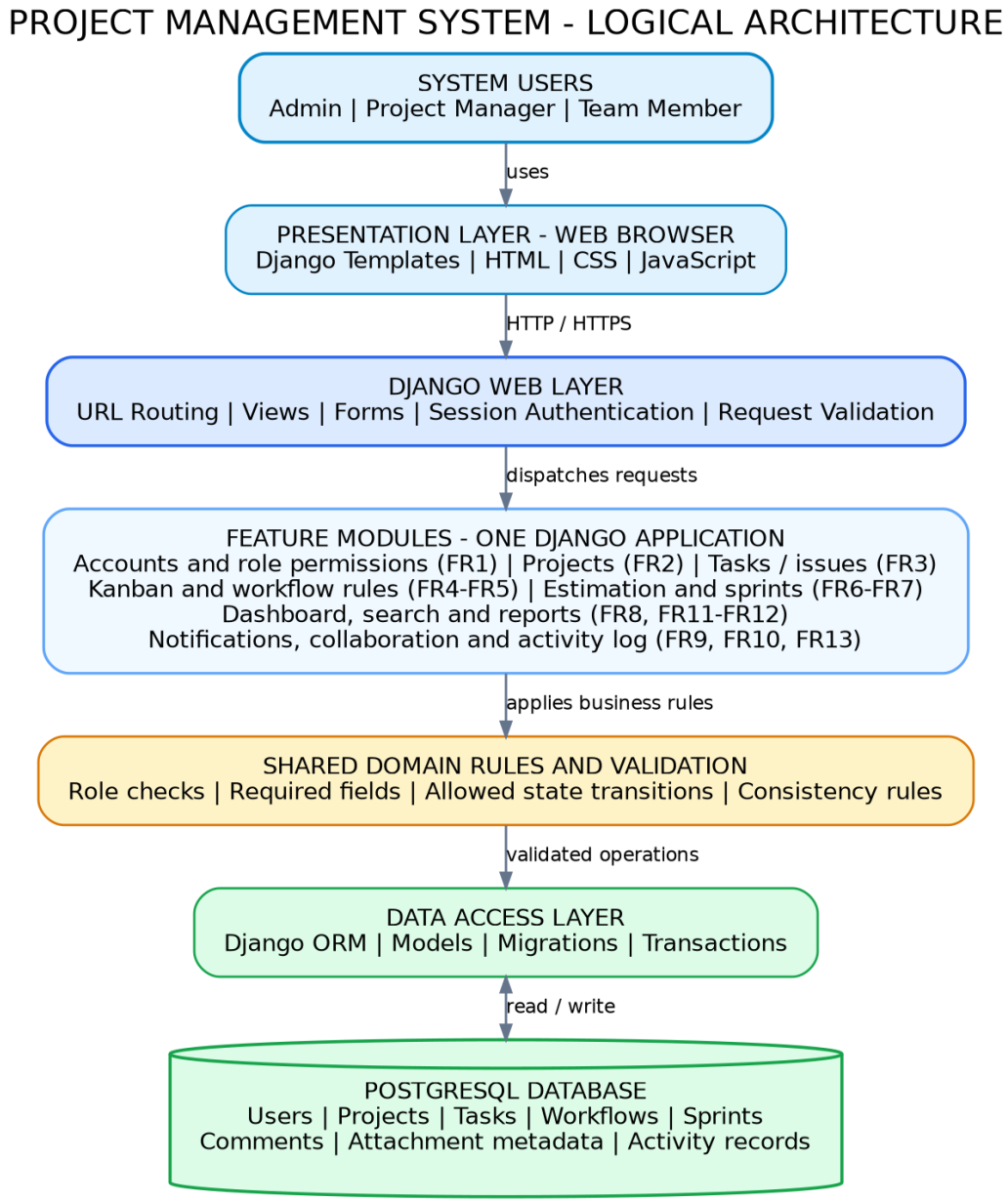


Figure 1. Proposed logical architecture of the Project Management System.

The runtime flow begins with the user in a web browser. Requests reach Django over HTTP/HTTPS, where authentication, authorization and input checks are applied. The relevant feature module executes business rules and uses the Django ORM to read or update PostgreSQL. The application then returns a page or an appropriate error response.

Implementation boundary: server-rendered Django pages are the starting approach. JSON/REST endpoints may be introduced where they provide a clear benefit. The final database tables, attributes, keys, relationships and indexes are documented separately in the database schema design.

3. Component Responsibilities

The following components define logical responsibilities and show how the architecture maps to the requirements. They can be organized as Django apps or modules according to the final repository structure.

Component	Primary responsibility	Requirement coverage
Presentation and views	Pages, forms, navigation, user feedback and browser responses.	UI across the system
Accounts and access	Registration, login/logout, session handling and role/permission checks.	FR1
Project management	Create, view and update projects; manage membership and lifecycle according to approved requirements.	FR2
Tasks and issues	Create, view, update, assign and track task/issue details.	FR3
Kanban and workflow	Display task states and validate allowed state transitions.	FR4-FR5
Estimation and sprint planning	Record estimates and organize sprint/milestone work and backlog membership.	FR6-FR7
Dashboard, search and reports	Present status and metrics, filter/search records, and generate reports.	FR8, FR11-FR12
Collaboration and activity	Comments, attachment metadata, notifications and important-action records.	FR9, FR10, FR13
Shared validation and rules	Apply authorization, required-field checks, business rules and consistent error handling.	Cross-cutting
Persistence	Use the ORM and database migrations to store and retrieve application records.	All features

3.1 Core data concepts

The SRS identifies these core concepts: User, Project, Task, Workflow, WorkflowState, Sprint, Comment, Attachment and ActivityLog. They inform module ownership and data flow, but this architecture document does not define their detailed fields, primary/foreign keys, cardinalities, indexes or database constraints. Those details belong in the database schema design.

3.2 Architecture boundaries

- Feature modules should call shared validation and access-control logic rather than duplicating policy checks inconsistently.
- Database reads/writes should use the configured data-access approach; browser code should never connect directly to the database.
- External integrations and file storage choices should remain isolated behind appropriate interfaces if they are added later.

4. Core Request and Data Flow

A normal application action follows one predictable route: the browser submits input, Django checks the session and permissions, server-side validation is applied, feature-specific rules run, the ORM persists valid changes, and the application returns a result.

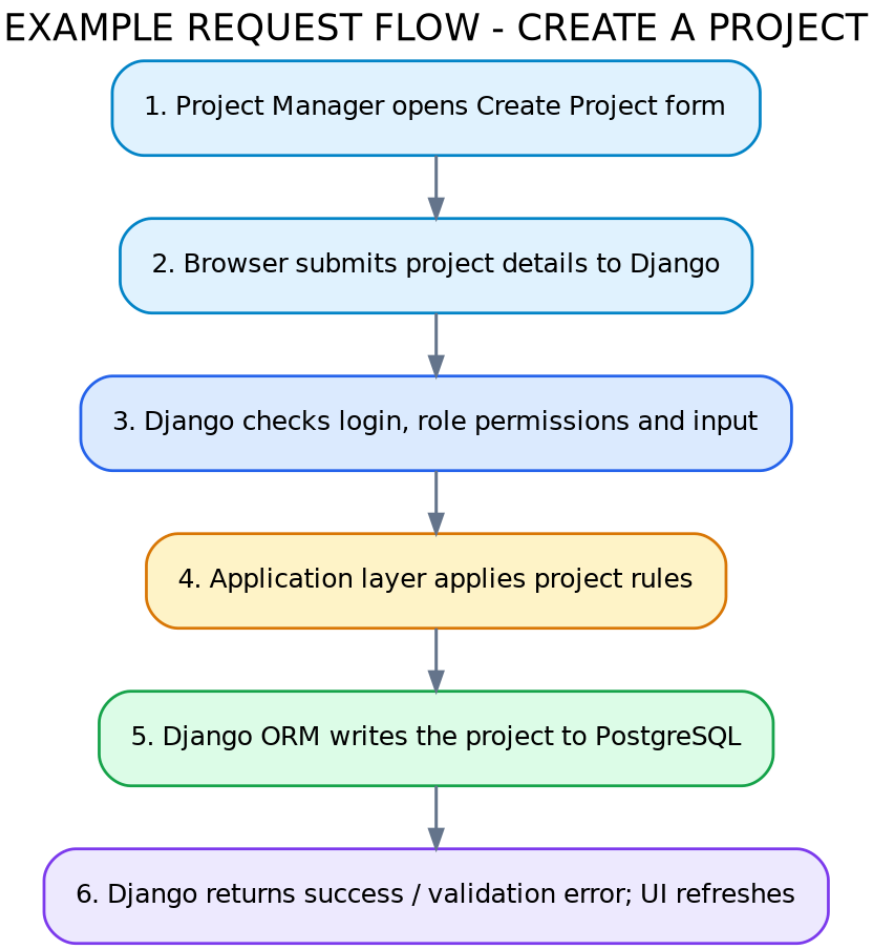


Figure 2. Example request flow for creating a project.

4.1 Example interaction: create a project

1. The Project Manager opens the project creation form in the browser.
2. The browser submits the entered project details to Django.
3. Django checks the user session, role permissions and required input.
4. The project management module applies the approved project rules.
5. The ORM saves the valid record to the database.
6. The application returns a success response or a validation message; invalid or unauthorized requests must not be saved.

4.2 Other interactions

- Login: validate credentials, establish the user session and check permissions on subsequent requests.
- Create task: validate project context and required details, save the task, and display it in the relevant view.
- Change Kanban state: validate the requested transition and reject unsupported transitions.
- Notifications and activity: record relevant events after successful operations when those features are implemented.

5. Security, Reliability and Quality Considerations

The controls below are architecture requirements and verification ideas. Their inclusion in this document does not indicate that they have already been implemented or tested.

Concern	Architecture control	Verification idea
Authentication and roles	Use Django authentication/session facilities and check permissions on every protected server-side action.	Valid login succeeds; invalid credentials fail; unauthorized actions are rejected.
Password and session safety	Use framework password hashing; never store plaintext passwords. Apply secure cookie and HTTPS settings when hosted.	Review authentication settings and verify that protected pages require login.
Input and request protection	Validate input on the server and use Django CSRF protection for state-changing browser requests.	Invalid or missing fields are rejected; forged state-changing requests are not accepted.
Data integrity	Use ORM operations, appropriate relational constraints and transactions where related records must change together.	Invalid references are rejected and failed operations do not leave partial updates.
Error handling	Return understandable user-facing errors and log technical details without exposing stack traces to users.	Expected errors produce controlled responses without corrupting stored data.
Performance and usability	Paginate larger lists, avoid repeated database queries, and provide clear navigation and validation feedback.	Check representative list/search operations and confirm clear validation messages.
Maintainability and portability	Keep modules separate, use migrations, configure secrets outside source control and pin dependencies.	A teammate can follow setup instructions and run tests in a clean environment.
Backup and recovery	Document a database backup and restore procedure before a hosted/production release.	Demonstrate restore from a backup before production use.

5.1 Cross-cutting rules

- Never rely on browser-only validation; repeat required checks on the server.
- Enforce role permissions for each operation, not only by hiding interface controls.
- Keep dependency definitions and migrations under version control so each teammate can reproduce the development environment.
- Do not commit credentials, local environment files, virtual environments or real user data to the repository.

6. Development Workflow and Continuous Integration

The architecture includes a proposed automated-check flow to support team development. This diagram describes the intended process; the GitHub Actions workflow must still be configured and verified in the repository.

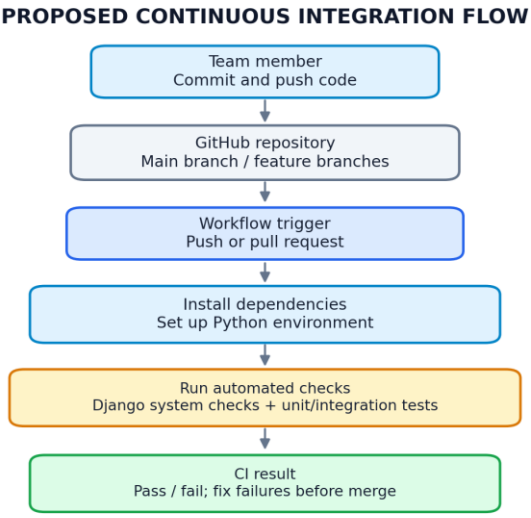


Figure 3. Proposed continuous-integration flow for the project repository.

6.1 Team collaboration model

- Each developer works on a local copy and commits small, identifiable changes using their own GitHub account.
- On a push or pull request, GitHub Actions should install dependencies, run Django system checks and execute available unit/integration tests.
- A failing automated check should be investigated and fixed before the affected changes are considered ready to merge.
- Jira tracks planned work and progress; GitHub stores source code, commit history and automation results.

6.2 Decisions to confirm before implementation

Decision	Current proposal / action
Technology stack	Confirm Python/Django, Django Templates and PostgreSQL as the team baseline before coding.
Frontend/API boundary	Start with server-rendered pages; add JSON/REST endpoints only where needed.
Attachment storage	Confirm whether local development storage is sufficient or hosted storage will be required later.
Hosting and deployment	Not selected in this design. Determine a target only when the team agrees on release needs.
Database schema	Define tables, attributes, primary/foreign keys, relationships, indexes and constraints in the database schema design.
Automated checks	Implement and verify GitHub Actions after the basic project structure and tests are available.

7. Summary

This proposed architecture provides a practical foundation for the Project Management System using clear application layers, separated feature responsibilities and a shared persistence boundary. The team should review and confirm the proposed stack and implementation boundaries, then build and test the system incrementally. Architecture diagrams and controls describe the intended design; completion should be claimed only after implementation and verification.